hwcost round 2

The ISA effect, the `fp64` branch lead, a GPU profile, and index calculus in time and dollars: results report, 2026-10-16

Study: `research/hwcost_round2_20261016/`; protocol `PROTOCOL.md` frozen at `597502b4d` before every run and analysis, addendum 1 frozen at `4e5994fc9` before its runs. **Run report:** `reports/run-20261016T1654Z/run_report.pdf` (every run, its host, gates and raw values). **Round 1:** `research/hwcost_model_20261016/`. **Code:** `src/cryptanalysis/hwcost/`, `src/bin/hwcost.rs` (Rust; Python only lays out graphs).

Scope. This round measures and prices; it changes no index-calculus pipeline and solves no discrete logarithm. It is **not** an index-calculus improvement and adds no gap-to-rho row (AGENTS.md success criterion). Part D compares IC with rho from frozen records and ranks two arms only where the wall-time order and the order at every §5a weight (0, 0.1, 1 GAE per unpriced native operation) agree (AGENTS.md §5a).

1. Findings

1. **ISA (part A).** A `-C target-cpu=native` build runs the `fp64` and `fp64bf` rho kernels 1.6–2.6× faster at every batch size (native/baseline 0.38–0.63 in both pairs of runs; 0.38–0.47 at B ≥ 256); the multi-limb `FpWide` kernels show no detected effect at 37 of 40 batch sizes. The model calibrated on the native build predicts it (I2 passes for every field), but a residual fitted on the baseline build does not transfer to the native one for `fp64`/`fp64bf` (I3 fails, error 1.3–1.5) while it does for `w1`–`w4` (≤ 0.10).
2. **Branches (part B, addendum 1).** The pre-registered test F1 was invalid: LLVM compiled `Fp64Bf`'s masks back into the same branchy code as `Fp64` (F3: identical 303 instructions, 32 conditional jumps). With the selections written as `select_unpredictable` (`Fp64Cm`: 20 jumps, 12 `cmov`), the kernel takes **0.5× or less** of `fp64`'s time at every B ≥ 4096 in both runs (F1′ PASS), and the a priori model, with no fitted parameter, predicts it within 8% and 4% (F2′ PASS) where it misses `fp64` by 54%. Round 1's `fp64` anomaly is the data-dependent branches in `Fp64::sub`/`redc`: they cost half or more of the kernel's time on random operands. Causal evidence by substitution; branch misses themselves could not be counted (no PMU on this host).
3. **GPU (part C).** With SMs as cores and the busiest pipe binding, the a priori profile predicts all 46 recorded ECC2K-130 builds within a median 17.9% (G1), never above the measured rate (no build with η > 1); one efficiency η per GPU brings it to 1.4% (G2); carried from the RTX PRO 6000 to the B200, 19.5% (G3). The cost arithmetic reproduces the paper's ECC2K-130 figure ($123,749 vs $124k).
4. **Index calculus (part D).** On every measured curve where §5a admits a rank, IC costs more wall time than the matched rho; on no curve does IC rank below rho. The prime band **understates** IC's wall cost from log2 r = 32 up (measured ratio above the band's whole range, 1.1–2.8×); the Koblitz band holds the measured ratio on its 3 largest curves. Both projections, the pre-registered wall-time fit and the post hoc GAE fit, rank IC > rho at every target from 2^{48} to 2^{128} and at every weight; in the post hoc projection IC costs 29 to 3.7×10^4 times rho's time (Koblitz) and 290 to 3.8×10^7 times (prime).

2. Part A: the ISA effect

Four full `measure` runs alternated baseline (A) and native (B) builds, A1 B1 A2 B2, and each arm was classified only when both ratios $r_k = B_k/A_k$ cleared $\max(2d, 2\%)$ on the same side, d the A1/A2 drift (claim I1).

arm group	native faster	native slower	no detected effect
clock	0	0	2
fp64	0	3	5
fp64bf	0	1	7
kernel	22	1	37
mem	9	1	10
scaling	9	0	9
w1	0	3	5
w2	1	0	7
w3	1	0	7
w4	1	3	4

The kernel arms called "faster" are all 20 `fp64` and `fp64bf` arms; among the 40 `FpWide` kernel arms, `w3` at B = 16 and `w4` at B = 64 are faster by 4–7% and `w1` at B = 65536 slower by 3–5%; every other one shows no detected effect. Of the 48 field-operation arms, 35 show no detected effect, 10 are slower on the native build (by 2% to 59%, the largest `fp64bf.add` latency) and 3 faster (by 2–12%): isolated operations got no faster, so the kernel speedup comes from the kernel's code generation (fewer branches, `mulx`, vector moves; table in part B). The memory-read arms are faster on the native build in 9 of 20 cases (ratios down to 0.43), so the read bandwidth this harness measures, and with it round 1's roofline constants, depends on the build.

field	V1 isa-base-1	V1 isa-base-2	I2 (V1) isa-native-1	I2 (V1) isa-native-2	I3 isa-native-1	I3 isa-native-2

fp64	0.042 PASS	0.008 PASS	0.09 PASS	0.071 PASS	1.506 FAIL	1.384 FAIL
fp64bf	0.023 PASS	0.032 PASS	0.081 PASS	0.05 PASS	1.321 FAIL	1.294 FAIL
w1	0.064 PASS	0.021 PASS	0.027 PASS	0.08 PASS	0.018 PASS	0.018 PASS
w2	0.068 PASS	0.042 PASS	0.12 PASS	0.075 PASS	0.015 PASS	0.094 PASS
w3	0.015 PASS	0.026 PASS	0.022 PASS	0.045 PASS	0.037 PASS	0.049 PASS
w4	0.063 PASS	0.029 PASS	0.077 PASS	0.045 PASS	0.029 PASS	0.022 PASS

Replication of round 1 on the baseline runs: C0 (clock) and V1 (calibrated model, test batches) pass for every field; V2 (a priori) fails for `fp64` (0.55, 0.54) and `fp64bf` as in round 1, and passes for `w1`–`w4`. On the native build V2 passes for `fp64` (0.145, 0.082): once the branches are gone, the op counts explain the kernel. I3′s failure for `fp64` says the same from the other side: the baseline residual (≈ 27 ns per iteration) is the branch cost, which the native build does not pay.

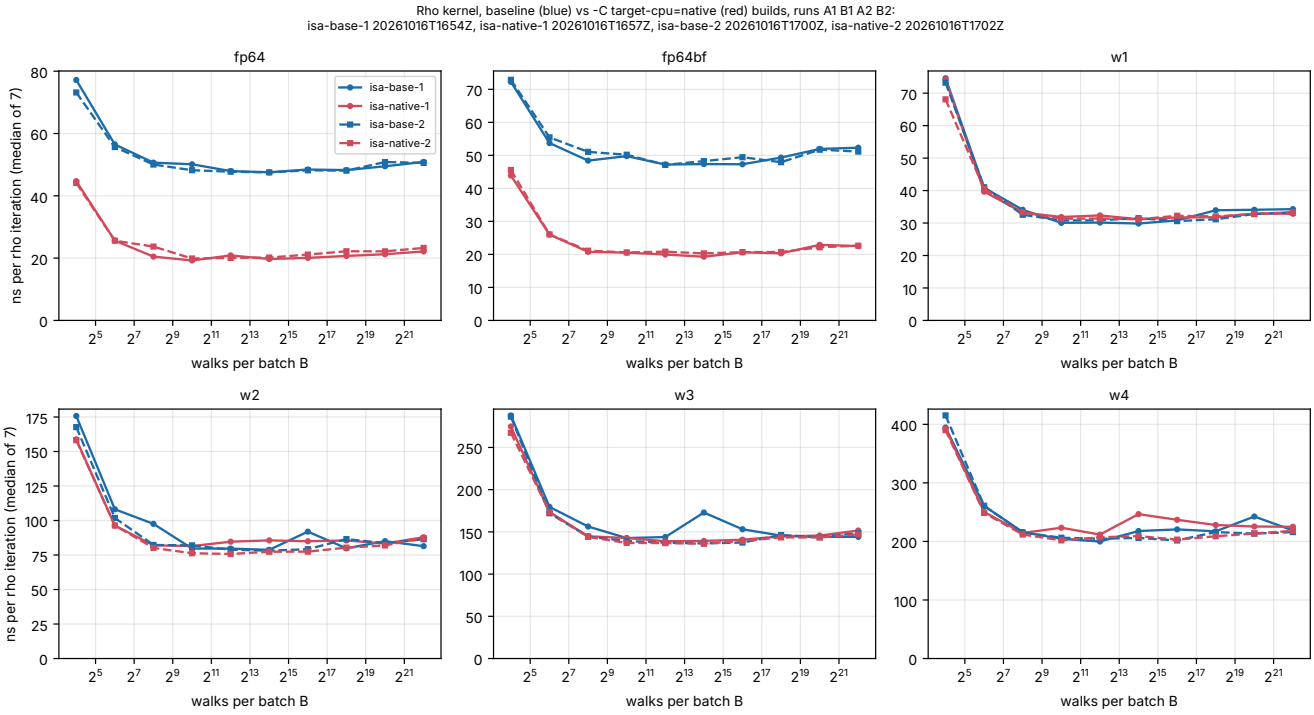


Figure 1: Rho kernel time per field, runs A1 B1 A2 B2 (medians of 7). Native (red) is lower only for `fp64` and `fp64bf`.

3. Part B and addendum 1: the `fp64` branch lead

Round 1 found the `fp64` kernel 1.6× slower than `w1`′s although its operations are cheaper in isolation, and named the data-dependent branches in `Fp64::sub` and `Fp64::redc` as an unverified lead. This host has no PMU, so the lead was tested by substitution: the same field, the same values, the branches replaced.

F1 was not a test. `Fp64Bf` wrote the two selections as masks. F1 failed (worst ratio 1.05), but F3 shows why: in the baseline binary the two kernels are the same instruction sequence. Addendum 1 replaced the masks with `std::hint::select_unpredictable` (`Fp64Cm`), checked the machine code **before** freezing, and re-tested.

build	kernel	instructions	cond. jumps	cmov	vector-register	mulx
baseline	`step_fp64`	303	32	4	0	0
baseline	`step_fp64bf`	303	32	4	0	0
baseline	`step_fp64cm`	285	20	12	0	0
native	`step_fp64`	337	22	14	38	12
native	`step_fp64bf`	337	22	14	38	12

claim	rule	value	verdict
F1	`fp64bf` ≤ 0.85 × `fp64`, every B ≥ 4096, A1 and A2	1.0499	FAIL (invalid: same code)
F2	V2 on `fp64bf` ≤ 0.30, A1 and A2	0.53, 0.551	FAIL
F1′	`fp64cm` ≤ 0.85 × `fp64`, every B ≥ 4096, cm-1 and cm-2	0.4998	PASS
F2′	V2 on `fp64cm` ≤ 0.30, cm-1 and cm-2	0.079, 0.043	PASS, PASS

`fp64cm / fp64` lies between 0.405 and 0.5 at the 12 cells with B ≥ 4096 (all batch sizes: 0.405–0.612). Model C's fitted residual per iteration drops from 27.2 ns (`fp64`) to −2 ns (`fp64cm`) in cm-1 (cm-2: 27 and −2 ns).

Reading. Only the selections differ between `fp64` and `fp64cm`, and the results are bit for bit equal (unit test and run gate). Removing 12 of 32 conditional jumps halves the kernel's time and makes it what its op counts predict; on random operands each of those branches is a coin flip. The native build's 2.1–2.6× `fp64` speedup at B ≥ 256 (part A) is therefore mostly the same effect reached by LLVM through `cmov` (22 jumps, 14 `cmov`), with `mulx` and vector moves on top; how the two shares split is not measured here.

Scope and limits. Measured on `hwcost::kernel`'s rho step (an r-adding walk with batched inversion), not on the repository's production rho or IC code; host L0 (shared cloud VM). Branch misses are inferred from the substitution, not counted.

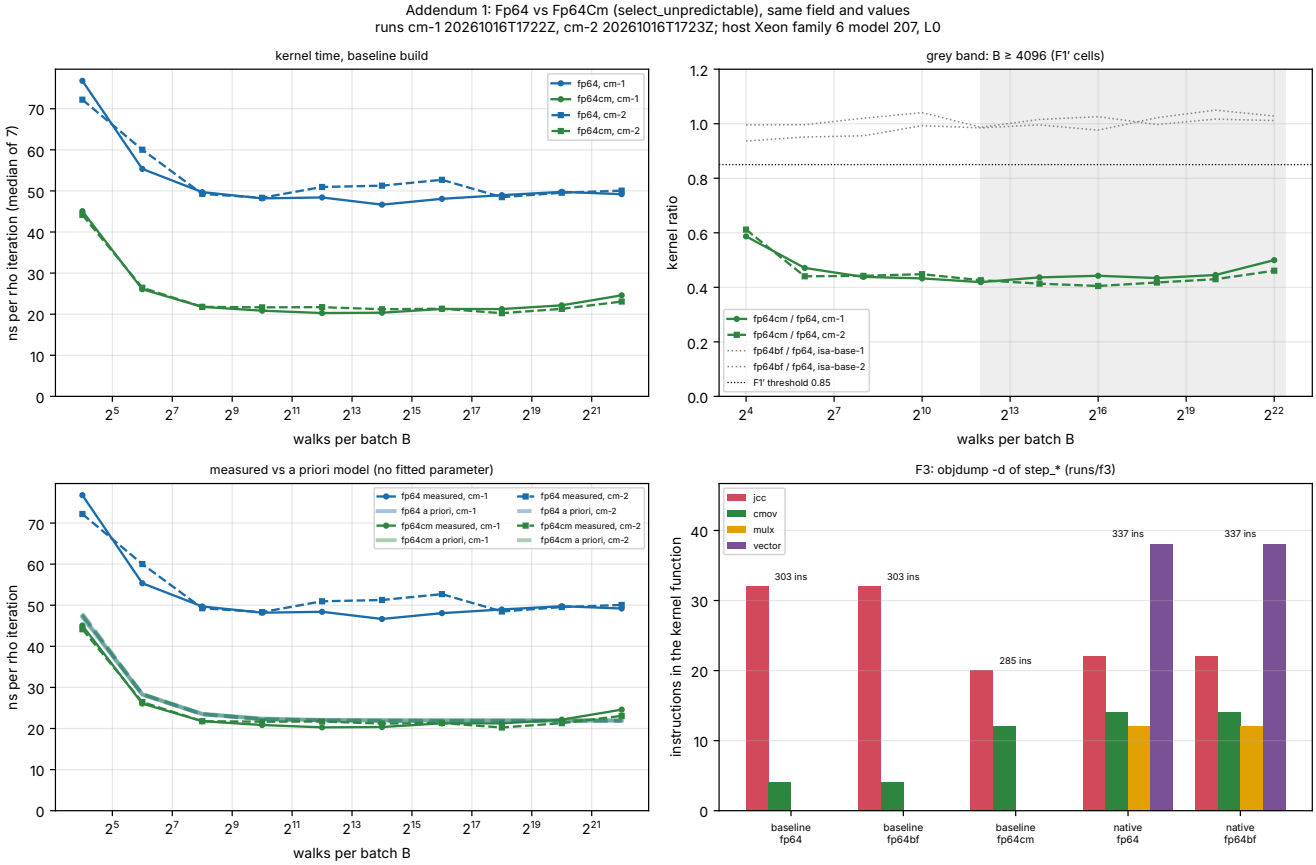


Figure 2: Runs cm-1 and cm-2: kernel time, the `fp64cm / fp64` ratio (part A's `fp64bf / fp64` dotted), measured against the a priori model, and the F3 instruction counts.

4. Part C: a GPU profile

Each of the 46 recorded ECC2K-130 builds (21 RTX PRO 6000, 25 B200; `ecc2k130/benchmarks/roofline/calibration.json`) has dynamic lane-instruction counts per update. The profile treats SMs as cores and an update's pipes as overlapping, so SM-clocks per update are the busiest pipe's count divided by its measured ceiling (`Phase::pipes_overlap`). Every constant has a source; issue width 128 and three B200 ceilings are flagged as assumed.

id	statement	value	threshold	verdict
G1	a priori, all 46 builds, median |error|	0.1792	0.3	PASS
G1.eta>1	builds measured above the a priori ceiling (η > 1)	0	—	REPORT
G2	η from each GPU's base build, 44 other builds, median |error|	0.0145	0.3	PASS
G2.10	the same, against 10%	0.0145	0.1	PASS
G3	η from the RTX PRO 6000 base build carried to 25 B200 builds, median |error|	0.195	0.3	PASS
G1.shared	shared-int pipes: a priori, median |error|	0.1237	—	REPORT
G1.shared.eta>1	shared-int pipes: builds with η > 1	0	—	REPORT
G2.shared	shared-int pipes: η per GPU, median |error|	0.0192	—	REPORT
G3.shared	shared-int pipes: η carried across GPUs, median |error|	0.1072	—	REPORT

The ceiling model is never beaten (no η > 1), as a ceiling should be. The one-parameter-per-GPU model is good to about 1.5%; the shared-integer-datapath variant fits better a priori (12%) and across GPUs (11%). Cost check: `expected ECC2K-130 work 2^60.9 at 14.63753 B/s on rtx-pro-6000 ($3.031/h): 40827.8 h, $123749`. The paper states $124k.

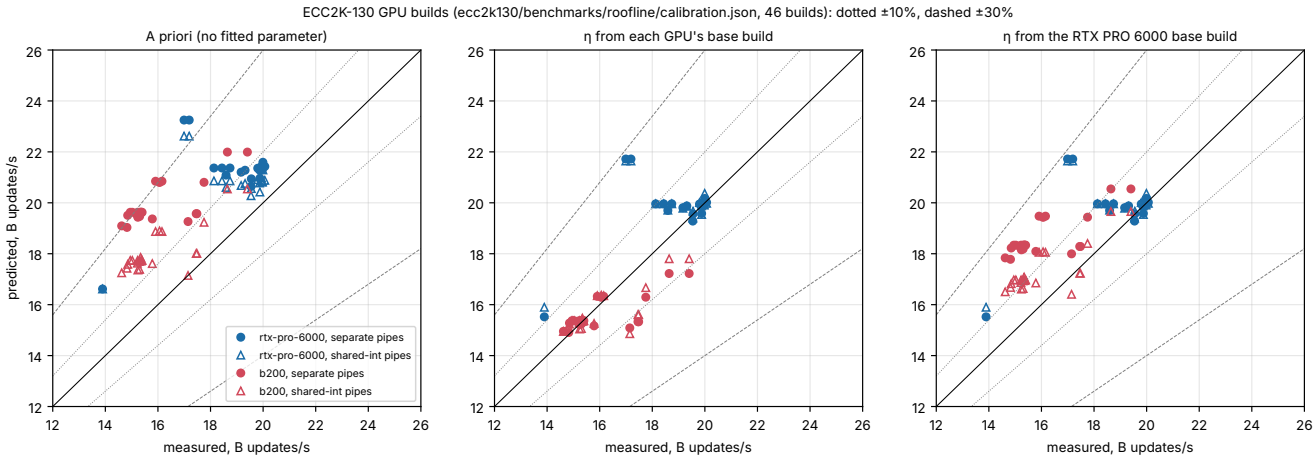


Figure 3: Predicted against measured update rate for the 46 builds, separate (filled) and shared-int (open) pipe models.

5. Part D: index calculus in time and dollars

Frozen Phase 1 records of `research/ic_frontier_campaign_20261015` (32 verified runs per arm and curve, host Xeon 2.30 GHz, L0): **Koblitz** `ic-compact` vs `rho-frob`, **prime** `ic-pair` vs `rho-neg`. The band prices each arm's work $P(w)$ at $w \in \{0, 0.1, 1\}$ GAE per unpriced native operation; w^* is the weight at which the band's ratio equals the measured wall ratio.

koblitz

curve	log2 r	IC ms	rho ms	wall ratio	band w=0	band w=1	w*	§5a rank
K13	11	0.15	3.02	0.049	6.87	27.1	−0.254	unranked
K17	16	0.25	3.07	0.081	3.8	11.85	−0.28	unranked
K19	18	0.32	3.25	0.099	4.88	23.6	−0.165	unranked
K23	22	1.06	3.57	0.296	5.48	18.45	−0.226	unranked
K37	27.8	4.35	4.15	1.049	5.2	23.29	−0.111	IC > rho
K43	32.1	22.77	3.03	7.504	7.35	23.14	0.005	IC > rho
K47	36.6	141.39	13.44	10.52	9.92	25.28	0.02	IC > rho
K41	39	498.9	25.69	19.424	15.02	30.3	0.169	IC > rho

D1.in_band = 3; D1.w_star_median = −0.138; D1.rank.IC>rho = 4; D1.rank.IC<rho = 0; D1.rank.unranked = 4.

prime

curve	log2 r	IC ms	rho ms	wall ratio	band w=0	band w=1	w*	§5a rank
P20	20	2.44	3.71	0.658	9.82	13.51	−0.615	unranked
P24	24	8.89	4.44	2.003	12.09	12.89	−0.929	IC > rho
P28	28	59.49	6.66	8.93	18.59	18.86	−0.966	IC > rho
P32	32	544.24	11.68	46.604	40.87	41.08	−1.085	IC > rho
P36	36	4913.35	51.48	95.435	58.15	57.48	−0.967	IC > rho
P40	40	63517.38	171.71	369.907	95.22	131.41	−1.359	IC > rho

D1.in_band = 0; D1.w_star_median = −0.967; D1.rank.IC>rho = 5; D1.rank.IC<rho = 0; D1.rank.unranked = 1.

Reading. On small curves rho's wall time sits on a per-solve floor of about 3–4 ms (setup, not iterations), so IC's wall time is lower on the smallest ones (Koblitz log2 r ≤ 22, prime 20) while every band weight ranks IC above rho; §5a leaves those curves unranked. From log2 r ≈ 28 (Koblitz) and 24 (prime) every curve ranks IC > rho. On Koblitz the three largest curves fall inside the band (w^* 0.005–0.17): the band's weights describe IC's wall cost there. On prime the measured ratio leaves the band from above at log2 r 32, 36 and 40 (by 1.1×, 1.6×, 2.8×): even $w = 1$ charges IC's unpriced work too little, and w^* is meaningless (the prime band's ratio barely moves with w).

Projections (extrapolated, not measured). The pre-registered projection fits $\log_2 y = a + \alpha \log_2 r$ to wall time over the tier. Rho's floor flattens its wall exponent (α ≈ 0.26 Koblitz, 0.34 prime, against 0.46–0.47 for its GAE count), so that projection

understates rho at large r by orders of magnitude; the post hoc projection (named in addendum 1) uses the fitted $P(w)$ exponents at rho's measured ns per GAE instead.

domain	log2 r	pre-reg. IC core-yr	pre-reg. rho core-yr	post hoc IC core-yr (w=0…1)	post hoc rho core-yr	rank
koblitz	48	5.88×10^{-7}	3.13×10^{-9}	3.96×10^{-7} – 4.95×10^{-7}	1.36×10^{-8} – 1.44×10^{-8}	IC > rho
koblitz	64	4.65×10^{-4}	5.31×10^{-8}	1.22×10^{-4} – 2.74×10^{-4}	2.17×10^{-6} – 2.53×10^{-6}	IC > rho
koblitz	80	3.68×10^{-1}	9.01×10^{-7}	3.01×10^{-2} – 1.82×10^{-1}	3.46×10^{-4} – 4.43×10^{-4}	IC > rho
koblitz	96	2.92×10^{2}	1.53×10^{-5}	7.43×10^{0} – 1.2×10^{2}	5.53×10^{-2} – 7.77×10^{-2}	IC > rho
koblitz	112	2.31×10^{5}	2.6×10^{-4}	1.83×10^{3} – 7.97×10^{4}	8.83×10^{0} – 1.36×10^{1}	IC > rho
koblitz	128	1.83×10^{8}	4.41×10^{-3}	4.53×10^{5} – 5.28×10^{7}	1.41×10^{3} – 2.39×10^{3}	IC > rho
prime	48	1.39×10^{-4}	2.65×10^{-8}	1.91×10^{-5} – 2.57×10^{-5}	6.67×10^{-8} – 6.83×10^{-8}	IC > rho
prime	64	9.81×10^{-1}	1.12×10^{-6}	2.85×10^{-2} – 4.84×10^{-2}	1.21×10^{-5} – 1.28×10^{-5}	IC > rho
prime	80	6.94×10^{3}	4.72×10^{-5}	4.25×10^{1} – 9.1×10^{1}	2.2×10^{-3} – 2.41×10^{-3}	IC > rho
prime	96	4.92×10^{7}	1.99×10^{-3}	6.35×10^{4} – 1.71×10^{5}	3.99×10^{-1} – 4.54×10^{-1}	IC > rho
prime	112	3.48×10^{11}	8.41×10^{-2}	9.48×10^{7} – 3.22×10^{8}	7.24×10^{1} – 8.53×10^{1}	IC > rho
prime	128	2.46×10^{15}	3.55×10^{0}	1.42×10^{11} – 6.05×10^{11}	1.31×10^{4} – 1.6×10^{4}	IC > rho

Dollars per solve at one vCPU of an illustrative c7i.xlarge ($0.1785/h ÷ 4) are in `projections.csv` and `projections_gae.csv`. At 2^{128} the post hoc projection prices a Koblitz rho solve at about 1.4–2.4 × 10^3 core-years and IC at 4.5 × 10^5 to 5.3 × 10^7; on prime, rho 1.3–1.6 × 10^4 and IC 1.4–6.1 × 10^{11}. The ranks are the same under both projections and every weight: IC > rho at every target.

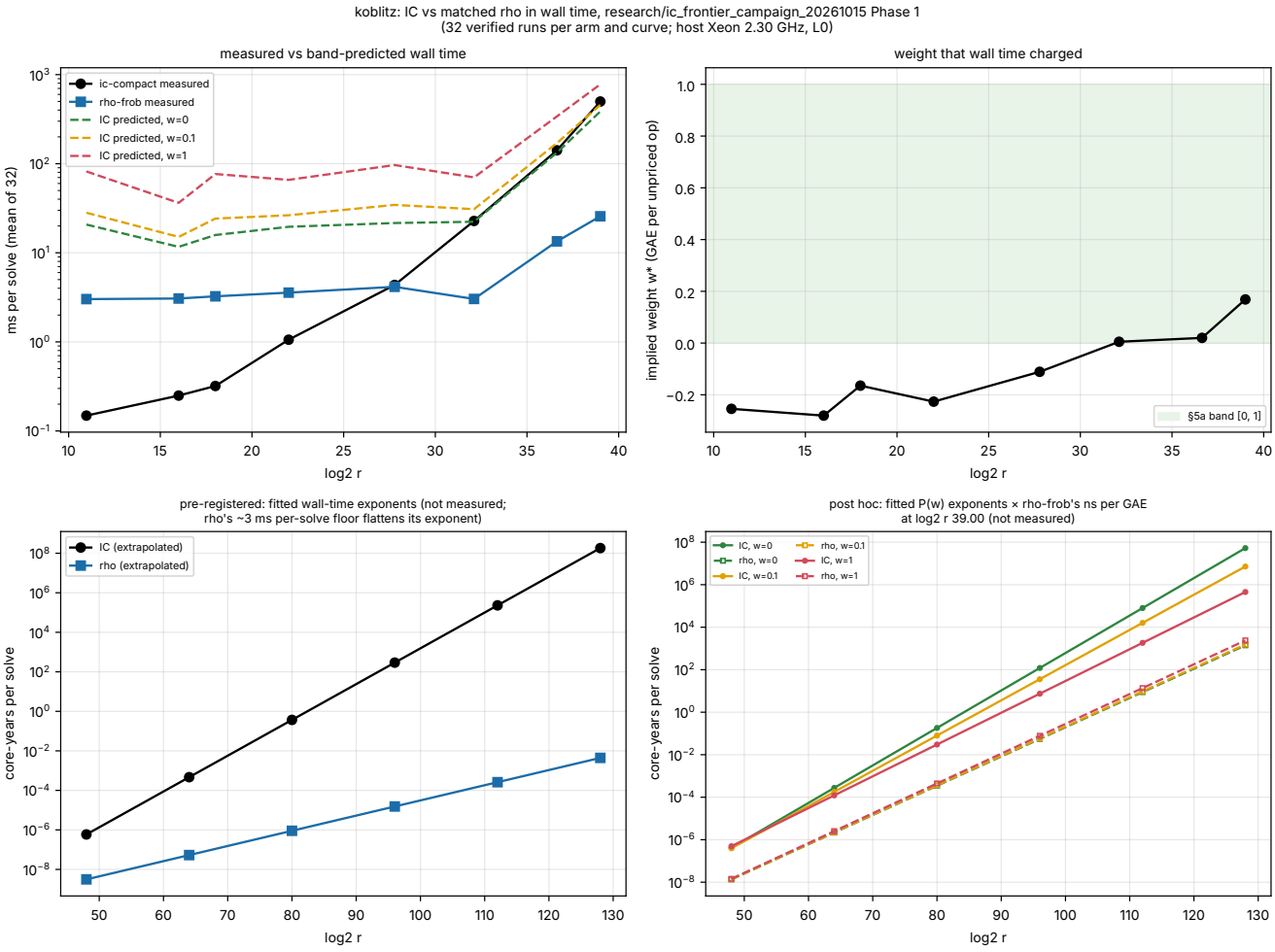


Figure 4: Koblitz: measured and band-predicted wall time (mean of 32 per arm and curve), implied weight, and both projections.

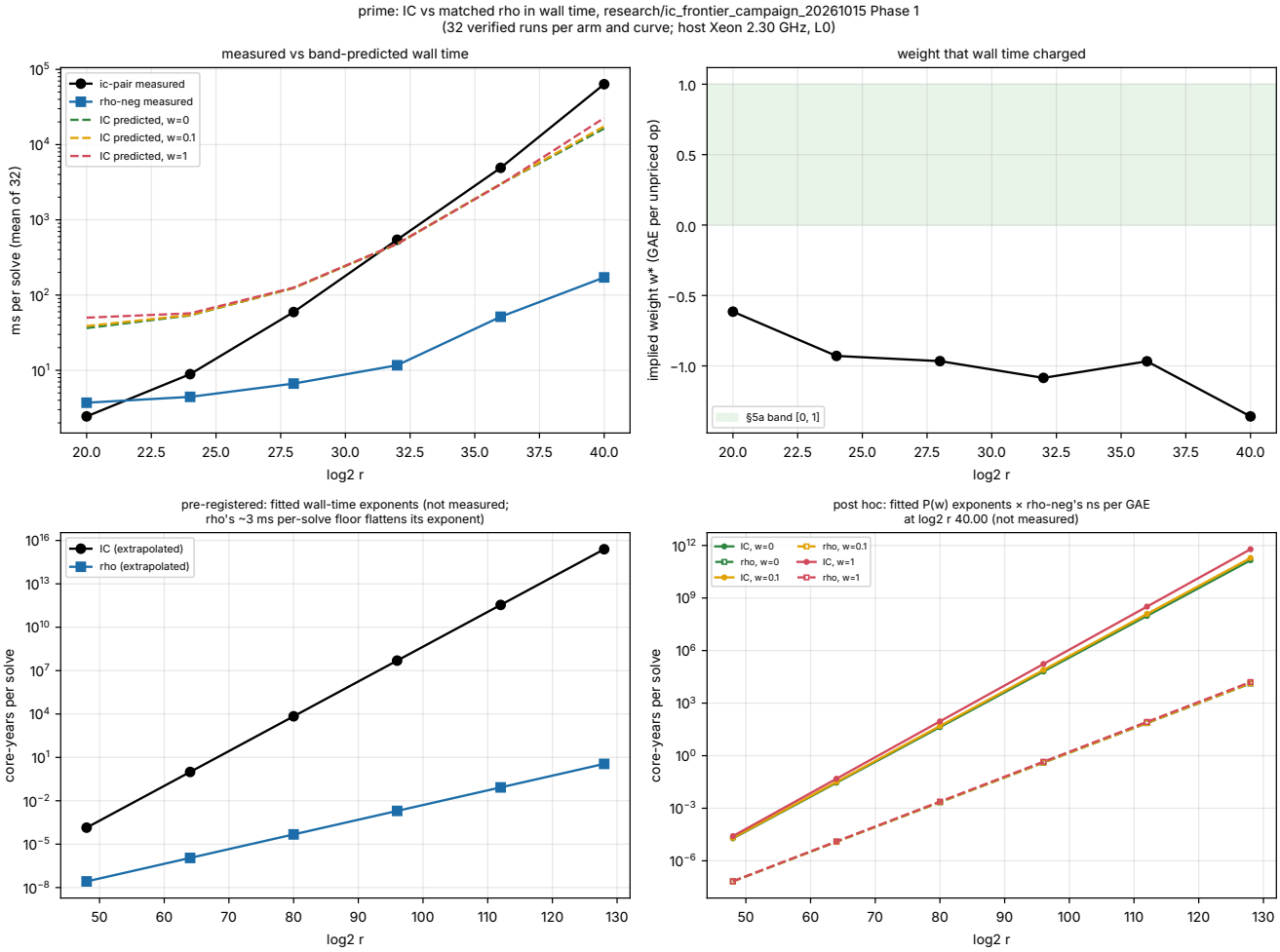


Figure 5: Prime: as above.

6. Claims table

id	rule	value	verdict
I1	per-arm classification (descriptive)	—	counts above
I2	V1 ≤ 0.30, native build, every field, B1 and B2	max 0.12	PASS
I3	baseline residual on native, ≤ 0.30, every field	`fp64` 1.51, `fp64bf` 1.32	FAIL (`w1`–`w4` pass)
F0	`fp64bf`, `fp64cm` walks equal `Fp64`'s	—	PASS (all runs)
F1	`fp64bf` ≤ 0.85 × `fp64`	1.05	FAIL; invalid test (F3)
F2	V2 on `fp64bf` ≤ 0.30	0.53	FAIL
F3	disassembly (descriptive)	—	`fp64` = `fp64bf` code
F1′	`fp64cm` ≤ 0.85 × `fp64`	0.5	PASS
F2′	V2 on `fp64cm` ≤ 0.30	0.079	PASS
G1	a priori, 46 builds, ≤ 0.30	0.179	PASS
G2	η per GPU, ≤ 0.30 (and ≤ 0.10)	0.0145	PASS (PASS at 10%)
G3	η carried across GPUs, ≤ 0.30	0.195	PASS
D1	in band; w^* (descriptive)	Koblitz 3/8, prime 0/6	—
D2	projections and §5a rank (descriptive)	—	IC > rho at every target

7. Limits

- Host L0: a shared 4-vCPU cloud VM without pinning, frequency control or a PMU. Same-build drift is in every margin; branch misses are inferred, not counted.
- Parts A and B measure `hwcost`'s own rho step, not the production rho or IC.
- Part C validates against recorded GPU rates; no GPU ran in this study.
- Part D's band and wall times come from another host (2.30 GHz) and campaign; both projections extrapolate exponents fitted on 4–5 curves over 13–18 bits of log2 r to 128 bits.

- The per-curve §5a `rank` column and `projections_gae.csv` were added after the analysis; they change no pre-registered value.

8. Next steps

1. Port the `select_unpredictable` selections to the production `fp_backend::Fp64` in its own PR, with a frozen baseline and the rho benchmarks as the gate: a candidate 2× on the baseline-ISA `Fp64` kernel.
2. Re-measure parts A and B on a host with a PMU to count branch misses and split the native speedup between `cmov` and `mulx`/vector code.
3. Price IC's memory-bound phases (sieve, linear algebra) with the roofline, so the band's unpriced native work gets a measured weight per phase.